Exercise 06

Task 1: Reference Counting

One of the shortcomings of Reference counting that was presented in the lecture is the inability to handle cyclic references.

1. Can a cyclic reference scenario occur in the SRCFLAE language? If so, give an example. If not, explain why it is not possible.

Consider:

```
Let('x, NewBox(10),
Let('y, NewBox('x),
Seqn(
  SetBox('x, 'y),
  OpenBox(OpenBox(OpenBox(OpenBox('x)))))))
```

Or

```
Let('x, NewBox(10),
Seqn(
  SetBox('x, 'x),
  OpenBox(OpenBox(OpenBox(OpenBox('x)))))))
```

2. Reference counting or variations are still used in modern programming languages. Can you think of ways to avoid or handle cyclic references in combination with reference counting? Can you imagine that reference counting and garbage collection could combined to have the advantages of both concepts?

Ideas:

- Disallow cycles by dynamic checks. Requires traversing all dependencies.
- One could use reference counting, but still do GC occasionally to detect cyclic structures. But while this has the advantages, that most memory is immidiately freed, and cyclic structures are detected, it also combines the overhead of both approaches.

3. Take a look at the following interpreter cases implemented for Reference counting. Explain in your own words why each call of `deref` and `enref` is required. Think about the possible consequences if some of the statements would be removed. Try to find examples that would produce memory leaks or lookup failures.

```
case SetBox(boxExpr, valueExpr) => {
  val (boxV, s1) = interp(boxExpr, stack, store)
  val (value, s2) = interp(valueExpr, stack, s1)
  boxV match {
    case Box(loc) =>
```

```
        // deref the old value in the box,
        // otherwise would leak memory
        val s4 = deref(loc, s2)
        // enref the new value we put in the box,
        // otherwise would fail lookup
        enref(value)
        (value, s4.update(loc, value))
      case _ => sys.error("can only set to boxes, but got: " + boxV)
    }
  }

  case Let(boundId, namedExpr, boundBody) => {
    val (namedVal, s1) = interp(namedExpr, stack, store)
    val (newLoc, s2) = s1.malloc(stack, namedVal)
    val (v, s3) = interp(boundBody,
        stack.head + (boundId -> newLoc) :: stack.tail, s2)
    // deref the location when the Id
    // goes out of scope, would leak memory
    val s4 = deref(newLoc, s3)
    (v, s4)
  }
```

4. Another issue of Reference counting is the memory overhead required to keep track of the counter. What do you think of the idea to use data types with a smaller capacity (like *short* or *byte*) to reduce memory usage?

Consider that smaller types may only allow a limited number of references.

Task 2: Non-Moving GC vs Moving GC vs Copying GC

Consider the following program:

```
def foo() {
  val x1 = Box(10)
  val x2 = Box(x1)
  val x3 = Box(x2)
  val x4 = Box(13)
  val x5 = Box(14)
  val x6 = Box(x4)
  val x7 = Box(x6)
  Box(x3)
}

val res = foo()
system.gc()
```

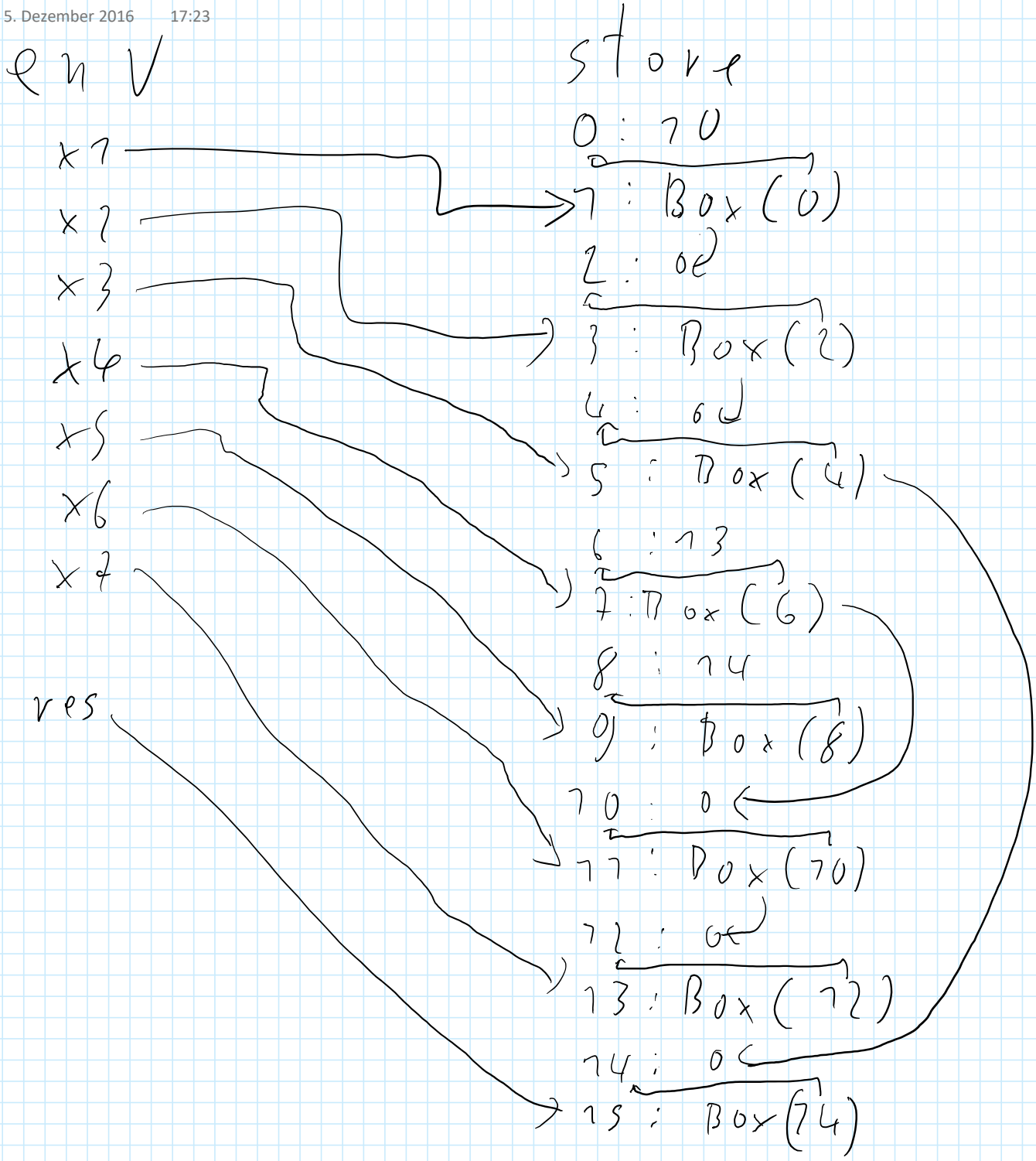
It calls method `foo` and then the garbage collector of our system.

Your task: Simulate the garbage collector run by modeling the heap before and after garbage collection. To this for three different GC strategies:

1. non-moving GC, as we implemented in the lecture
2. moving GC, which defragments the heap by collecting all non-released objects in one part of the heap
3. copying GC, which splits the heap in two and copies values from one half to the other

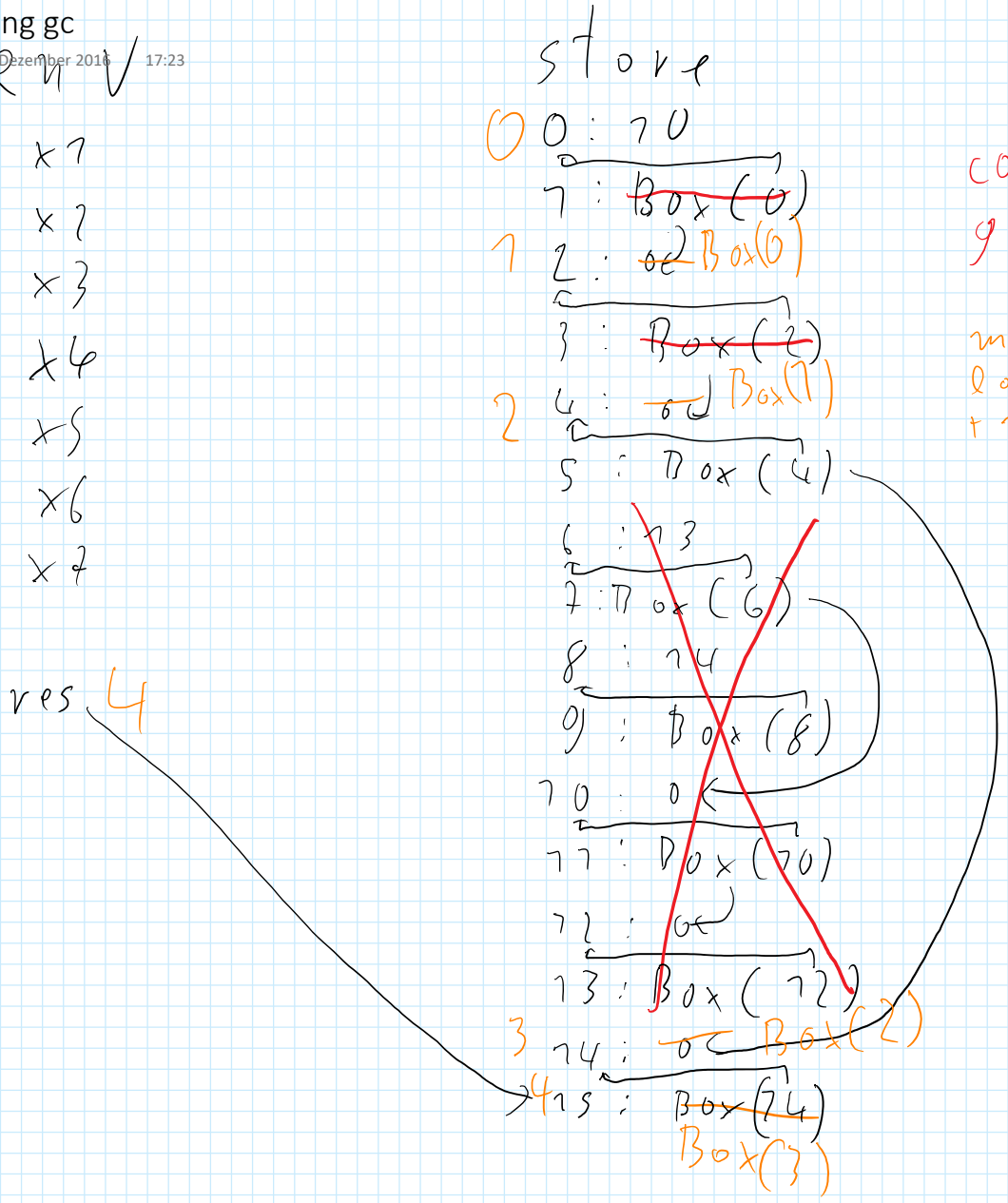
Initial Store

Montag, 5. Dezember 2016 17:23



non moving + moving gc

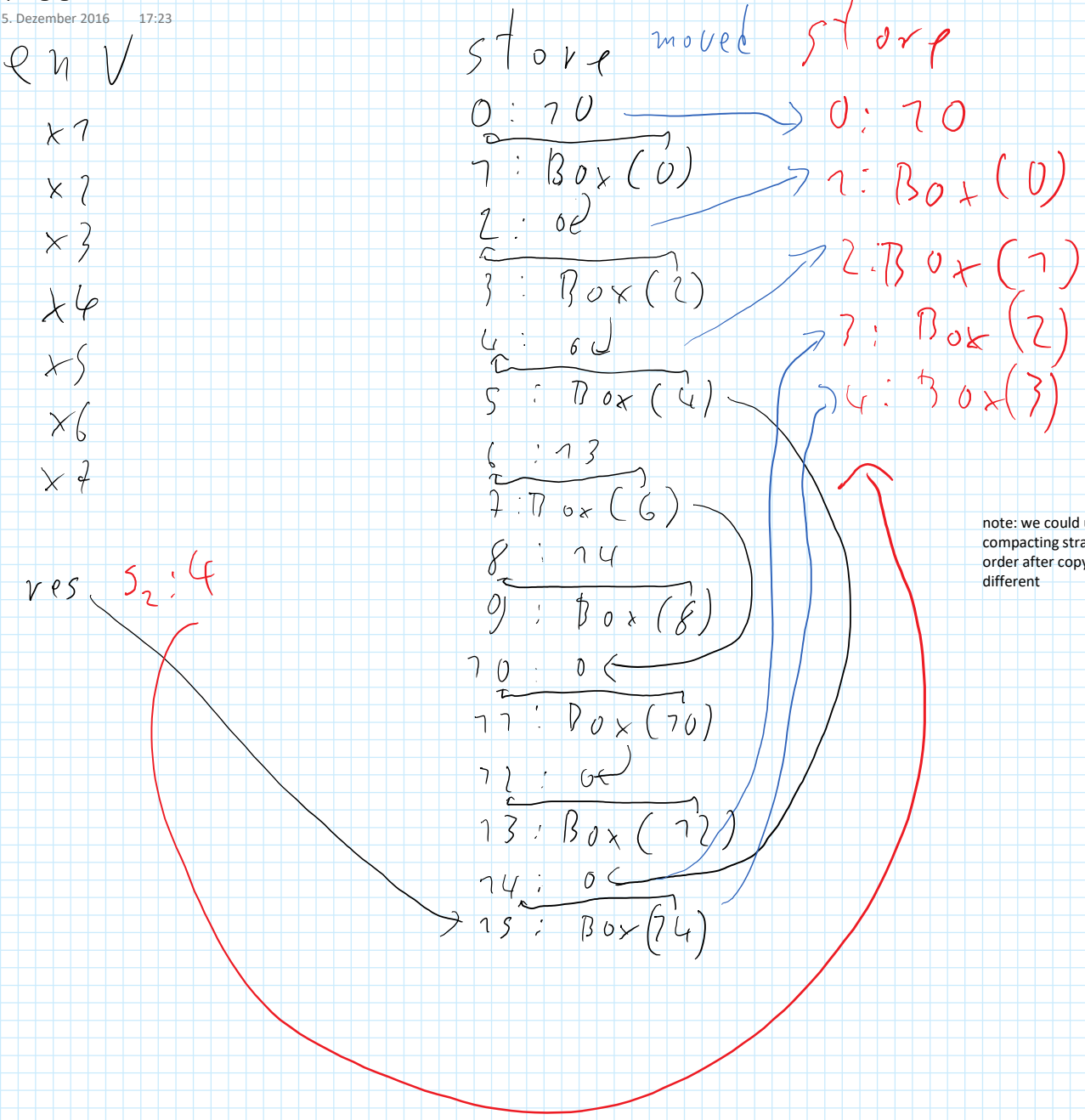
Montag, 5. Dezember 2016 17:23



note: we could use a different compacting strategy. i.e. store order after moving could be different

copying gc

Montag, 5. Dezember 2016 17:23



note: we could use a different compacting strategy. i.e. store order after copying could be different